

Incident Management System

INaAI Competition 2026 · Full-stack Developer Track · Fase 1 Proposal

1. Problem dan Solusi

Problem. Ketika terjadi production incident — server down, error rate melonjak, atau payment timeout — tim engineering sering menghadapi kekacauan koordinasi: siapa yang handle, sudah di-acknowledge belum, kapan harus eskalasi, dan bagaimana mencegah kejadian serupa? Tanpa sistem terpusat, informasi tersebar di Slack, email, dan grup WA, memperpanjang Mean Time to Resolution (MTTR) secara signifikan.

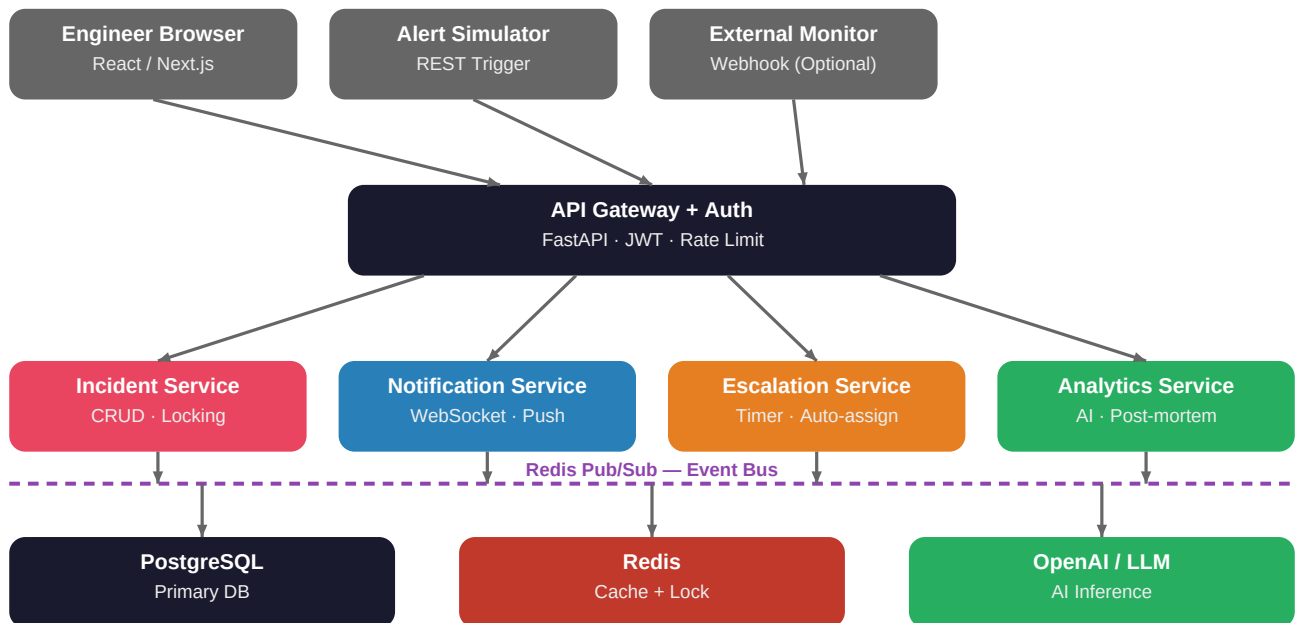
Solusi. Platform Incident Management System berbasis web yang mengelola siklus hidup penuh sebuah incident: deteksi → assignment → acknowledgement → resolusi → post-mortem. Sistem dirancang event-driven dengan tiga properti utama:

Pain Point	Solusi yang Dibangun
Dua engineer klaim incident bersamaan	Optimistic locking + atomic compare-and-swap di DB
Tidak ada yang merespons alert	Escalation Service dengan timer otomatis
Koordinasi tersebar di banyak channel	Dashboard real-time terpusat dengan WebSocket
Post-mortem manual dan lambat	AI auto-generate draft dari timeline incident
Tidak ada visibilitas pola incident	Analytics AI: clustering + prediksi recurrence

2. Tech Stack

Layer	Teknologi	Alasan
Frontend	Next.js 14 + TypeScript + TailwindCSS	SSR, type-safe, ecosystem matang
Backend	FastAPI (Python)	Async-native, auto OpenAPI docs, cepat
Database	PostgreSQL	ACID transactions untuk locking yang proper
Cache & Lock	Redis	Atomic ops (SET NX) untuk distributed lock
Event Bus	Redis Pub/Sub	Lightweight, cukup untuk skala kompetisi
Real-time	WebSocket (FastAPI native)	Push notifikasi ke browser tanpa polling
AI	OpenAI GPT-4o / Claude API	Generate post-mortem & analytics insight
Container	Docker + Docker Compose	Semua service reproducible dari clean state
CI/CD	GitHub Actions	Auto lint → test → build → deploy
Deploy	Railway / Render	Free tier cukup, Docker-native

3. Architecture Diagram



→ Synchronous REST/WS - - - Event Bus (async pub/sub)

Gambar 1. System architecture — 4 microservice terhubung melalui Redis Pub/Sub event bus.

Setiap service berjalan di container terpisah dan berkomunikasi secara async melalui event bus. **Incident Service** menjadi source-of-truth dan publisher utama. **Notification Service** consume event dan push ke browser via WebSocket. **Escalation Service** menjalankan timer per-incident dan mempublish event eskalasi jika tidak ada acknowledgement dalam threshold waktu. **Analytics Service** consume event resolved untuk membangun dataset dan memanggil LLM API guna generate post-mortem serta insight pola incident.

4. Roadmap Fase 2 (21,5 Jam)

#	Deliverable	Durasi	Kriteria
1	Docker Compose + skeleton 4 service + PostgreSQL schema	2 jam	Must Have
2	Incident CRUD, status machine, optimistic locking (Redis SET NX)	2.5 jam	Must Have + Extraordinary
3	Redis Pub/Sub event bus + event contracts antar service	1.5 jam	Must Have
4	Notification Service + WebSocket real-time ke frontend	1.5 jam	Must Have
5	Escalation Service dengan configurable timer + auto-assign	2 jam	Must Have
6	Frontend dashboard: incident list, detail, acknowledge, resolve	2.5 jam	Extraordinary
7	AI Analytics: post-mortem generator + incident clustering	2 jam	Extraordinary
8	GitHub Actions CI/CD + OpenAPI docs + structured logging	1.5 jam	Nice to Have
9	Redis caching untuk read-heavy endpoint + cache invalidation	1 jam	Nice to Have
10	TDD: unit test untuk locking logic + event flow sebelum impl	1 jam	Nice to Have
–	Buffer: polish UI, seed data historis, README, demo prep	4 jam	–

Total estimasi: ~21,5 jam. Buffer 4 jam dialokasikan untuk polish, seed data 100 incident historis untuk demo AI analytics, dan persiapan Live Defense.